The adaptive DIPF policy (v14)

How a 40-number learned policy retunes the potential field every 0.1 s, what it learned, and how it compares with the best fixed parameter set

1. Summary

The DIPF potential field drives the ego exactly as before: every acceleration is the clamped negative gradient of the potential. The adaptive policy changes four of the potential's parameters while the ego drives, based on nine measurements of the surrounding traffic. It was trained with CMA-ES on closed-loop segment success, on top of the **best fixed parameter set** that the same search could find (v14/base).

on 300 fresh test scenarios × 3 futures (900 paired runs)	**success**	**progress**
v12 library set 2 (the v13 base)	0.588	0.750
trained fixed set (v14/base, all 7 parameters tuned)	0.646	0.780
adaptive policy (v14/adaptive)	**0.714**	**0.812**

Adaptive vs the best fixed set: **+6.9 points** (runs only adaptive succeeded 182, only the fixed set 120; McNemar p = 0.0004). About one in five of the fixed set's failures becomes a success. The policy's offsets alone, without the adapting, are no better than the fixed set, so the gain comes from adapting during the run.

How to read this document. Sections 2 and 3 explain why adapting can help and what the four adapted parameters do inside the potential. Sections 4 to 6 describe the policy itself: where it runs, what it measures, and how it turns measurements into parameter values, with one control step worked through number by number. Sections 7 and 8 show what it learned and how it behaves in real runs. Sections 9 to 11 cover training, results on the training road and on a 1000 m road. A glossary is at the end.

2. Why adapt the parameters at all?

A fixed parameter set must work in every situation the ego meets, so it is a compromise. The same setting can be right in one situation and wrong in the next:

- **Strong repulsion** (large λ_{obs}) pushes the ego clear of a car cutting in from the side, but behind a slow car that is no threat it makes the ego brake or swerve for nothing.
- **A wide lateral reach** (large τ_y) keeps a safe margin to cars alongside at speed, but when the ego must swerve around an obstacle ahead, it also makes the free space beside it feel blocked.
- **A long look-ahead** (large obstacle radius) lets the ego react early in light traffic, but in a crowd it adds many distant vehicles whose forces only blur the picture.

Control engineers solve this with **gain scheduling**: the controller stays the same, and its gains are changed according to the operating condition. The adaptive policy is a learned gain schedule for the DIPF potential. It reads how threatening and crowded the situation is, and moves the four parameters up or down from the best fixed values. The search found the schedule by trial in simulation; nobody wrote the rules by hand.

Two design rules follow from the controller's safety story. First, the policy only changes *parameters*, never accelerations, so the vehicle is always driven by a physically limited gradient of a potential. Second, the policy is *small and bounded* (40 numbers, each parameter within ×0.1 … ×10 of the fixed set), so it can be read, checked, and cannot drift far from known-good behaviour.

3. Background: the DIPF potential and the four adapted parameters

The ego moves down the gradient of a potential with three parts: a pull towards the destination, walls at the road edges, and one repulsive hill per nearby vehicle. For a vehicle at offset (Δx, Δy) with relative velocity (Δv_x, Δv_y):

```
U_obs = λ_obs / k′
k′    = (shape term) · sqrt( (Δx/τ_x)² + (Δy/τ_y)² )
          shape term = 1 − size/|Δx| (mostly ahead/behind) or 1 − size/|Δy| (mostly
beside)
τ     = τ_base · [ (2 + α·Δv) + α·Δv·tanh(Δv) ] / 2       for τ_x (Δv_x) and τ_y (Δv_y)
U_obs = 0 if Δx > obs_radius_ahead, Δx < −20 m or |Δy| > 8 m
```

k′ is a scaled distance to the other vehicle: small when it is close, so the hill is high. Dividing by τ stretches the hill: a larger τ_y makes a vehicle feel closer sideways. The shape term accounts for both vehicles' sizes, so the hill rises steeply near body contact. Each adapted parameter changes the hill in a different way (Figure 1):

- **λ_{obs}** multiplies every obstacle hill. Doubling it doubles every repulsive force; the destination pull and road walls stay the same, so the balance tips towards keeping away from traffic.
- **τ_y** sets the lateral width of each hill. Larger: vehicles in the next lane push harder sideways. Smaller: the hill is narrow, so the ego can pass close beside a vehicle.
- **α** makes the hill grow with relative speed: a vehicle approaching fast gets a longer, wider hill. At α near 0 the hill depends only on position.
- **obs_radius_ahead** is a hard window: vehicles further ahead are ignored completely.

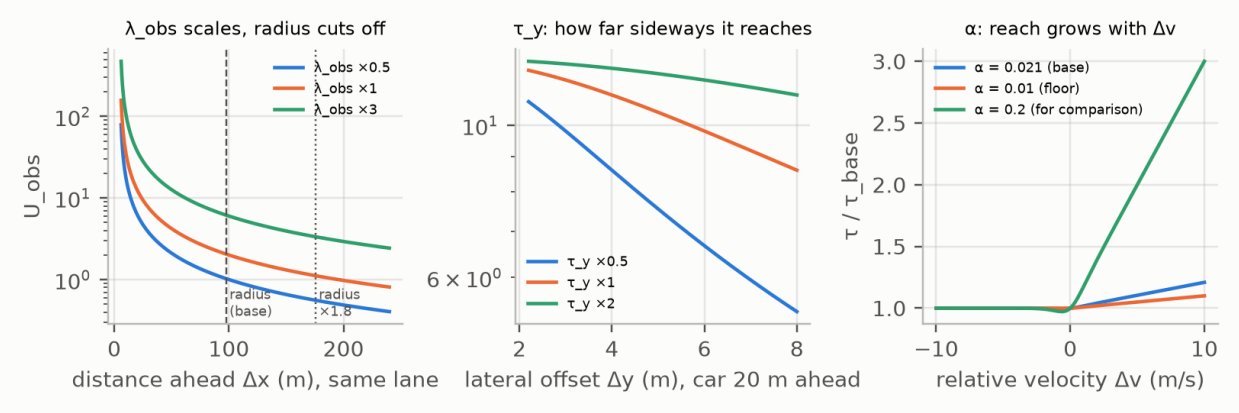


Figure 1. What the four parameters do, for a car (5 m × 2.1 m) at equal speed, with the trained fixed set's values. Left: the hill along the lane for three values of λ_obs, and where the look-ahead window ends. Middle: the hill sideways, 20 m ahead, for three values of τ_y. Right: the τ stretch factor versus relative speed for three values of α.

4. Where the policy sits in the controller

Every 0.1 s the controller computes $U = w_{dest}\cdot U_{dest} + w_{lane}\cdot U_{lane} + w_{obs}\cdot \Sigma U_{obs}$, its gradient, and $a = \text{clamp}(-\nabla U)$: +3.2 / −9 m/s² along the road, ±3 m/s² across, speed ≤ 34 m/s, sideways speed ≤ 3 m/s and ≤ $v_x\cdot\tan 15°$. The policy runs just before this and only sets parameters. It never outputs an acceleration, so a mistake can reshape the field but cannot command an arbitrary manoeuvre.

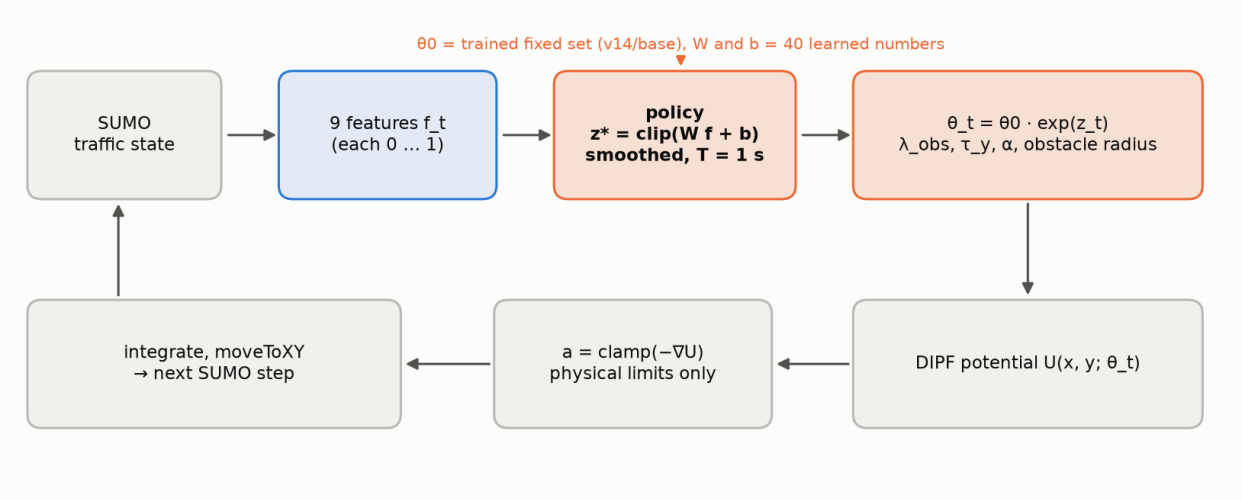


Figure 2. One control step. Orange: the policy; the rest is the unchanged DIPF controller.

Order within one 0.1 s step: (1) read all vehicles near the ego from SUMO; (2) build the traffic encoding and the nine features; (3) update the four parameters; (4) select obstacles with the *new* look-ahead radius; (5) evaluate the potential and its gradient with the new parameters; (6) clamp the acceleration, integrate velocity and position, and place the ego in SUMO. The parameters chosen at step t therefore act on the same step's traffic, without a one-step delay. In code this is `adapt_parameters()` in `v14/adaptive/dipf_controller.py`, called at the start of `ego_control_step()`; the maths is in `policy.py`.

5. Inputs: nine dynamics features

Where the numbers come from. Every step the traffic within 150 m is described by an 80-value encoding (`encoding.py`). It uses no lanes, because bikes filter between lanes and traffic is sub-lane. Instead the space around the ego is split into 8 sectors relative to its own body and path: front, front-left, front-right, left, right (alongside), rear, rear-left and rear-right. In each sector only the most critical vehicle is kept, described by 9 values (gaps between the two bodies, relative velocity, inverse time-to-contact, required braking, size). Eight global values add the ego's speed, heading, road-edge clearances and traffic density.

Time to contact is computed between the two bounding boxes, assuming both keep their current velocities: it is the first time the boxes would overlap, in x and y together. A car beside the ego that is drifting towards it therefore has a finite TTC, even with no longitudinal closing. **DRAC** (deceleration rate to avoid crash) is the braking that would just avoid contact with the car ahead: closing speed² / (2 × gap).

The 80-value traffic encoding is condensed into nine numbers, each scaled to [0, 1]. They describe how threatening or crowded the situation is, not where vehicles are (the potential already handles direction), and none of them changes when left and right are swapped, matching the symmetric controller.

feature	definition	large (→1) when
ttc_front	$1 - e^{-1/\mathrm{TTC}}$, vehicle in the ego's path ahead	closing in fast ahead (TTC < 1 s)
ttc_side_front	same, worst of front-left / front-right	closing in on a diagonal vehicle
ttc_rear_side	same, worst of rear, rear-left/right, alongside	something closing from behind or the side
drac_front	$1 - e^{-\mathrm{DRAC}/3}$, DRAC = closing² / (2·gap)	hard braking needed
gap_front	$e^{-\mathrm{gap}/20\ \mathrm{m}}$ (0 if nothing ahead)	a vehicle close ahead
gap_side_front	same, closest of front-left / front-right	a vehicle close diagonally ahead
tight_lateral	$e^{-\mathrm{room}/1\ \mathrm{m}}$, room = smallest road-edge clearance or side gap	no room to swerve
speed	v_x / 34 m/s	driving fast

feature	definition	large (→1) when
density	$1 - e^{-n/10}$, n = vehicles within ±50 m	crowded

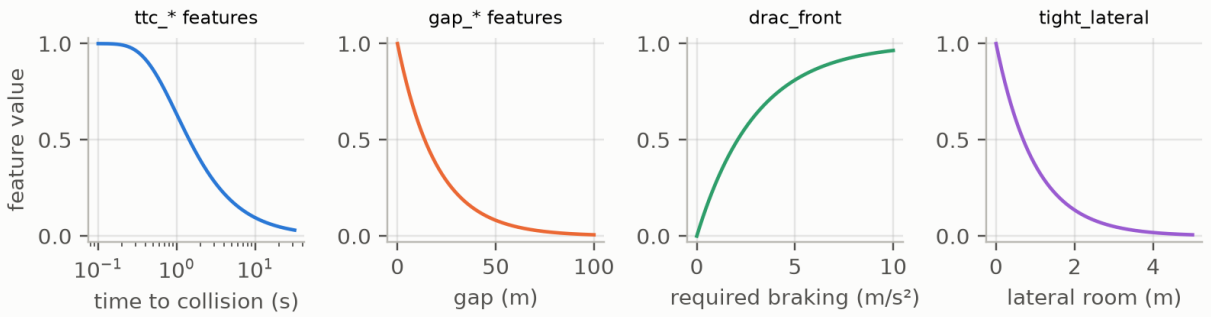


Figure 3. The transforms map unbounded quantities into [0, 1]; "nothing there" and "far away" both give 0, and each saturates where it stops mattering.

6. From features to parameters

The policy has three stages: a weighted sum gives a target exponent per parameter, smoothing filters it over time, and an exponential turns it into a multiplier of the fixed value.

```
z*_t = clip(W·f_t + b, −ln 10, +ln 10)      W: 4×9, b: 4 → 40 numbers
z_t  = z_(t−1) + (Δt/T)·(z*_t − z_(t−1))    T = 1 s, Δt = 0.1 s; first step z = z*
p_t  = clamp(p0·exp(z_t), limits)           p: λ_obs, τ_y, α, obstacle radius
```

Each adapted parameter gets one exponent: a weighted sum of the features plus an offset. The exponential makes changes multiplicative (natural for parameters spanning orders of magnitude) and keeps them positive; the clip limits each parameter to ×0.1 … ×10 of its base value; the absolute limits are the base search's ranges. Smoothing moves the exponent 10 % of the way to its target each step, so a parameter follows a sudden change within about a second instead of jumping. With W = 0 and b = 0 the policy reproduces the fixed set exactly (checked run by run).

adapted parameter	base p0 (trained fixed set)	limits	role in the potential
λ_obs (lambda_obs)	190.7	0.01 … 10000	strength of every obstacle's repulsion, U_obs = λ / k′
τ_y (tau_y_base)	0.3776	0.01 … 20	lateral scale in k′: larger → vehicles to the side repel more
α (alpha)	0.02091	0.01 … 100	how strongly τ grows with the relative velocity Δv
obs_radius_ahead	97.48 m	10 … 250 m	how far ahead obstacles are considered

The other parameters stay at the trained fixed set's values: s_x = 0.000992, δ_{lane} = 100, y_{margin} = 0.00542; τ_x = 1. Sign constraint: the weights from ttc_front, ttc_side_front and drac_front to λ_{obs} may not be negative, so repulsion can only grow with those dangers.

Why each design choice:

- **Linear in the features.** 40 numbers are enough to express "more of this danger → more of this parameter", can be read off directly (Figure 5), and are cheap to search without gradients. A small neural network would need many more numbers and much more simulation.
- **Exponent, not the parameter itself.** The parameters span orders of magnitude ($\lambda_{obs} \approx 190$, $\alpha \approx 0.02$). Working with log-multipliers treats ×2 and ×0.5 symmetrically for every parameter, keeps values positive, and lets one learning rate fit all four.
- **Clip to ±ln 10.** No combination of features can move a parameter more than ten-fold from the fixed set, which bounds how strange the field can become.

- **Smoothing.** Features can jump between steps (a vehicle enters a sector, the most critical vehicle changes). A jumping parameter would make the force jump and the ego jerk. First-order smoothing with T = 1 s turns a jump into a ramp (Figure 4): 63 % of the change after 1 s, 86 % after 2 s. At the takeover the smoothing starts at the target, so there is no ramp-up from the fixed values.
- **Offset b.** It shifts the parameter in calm conditions (all features ≈ 0). It is how the policy can choose a different 'calm' setting than the fixed set; the comparison 'offsets only' in Section 10 tests whether that alone explains the gain.

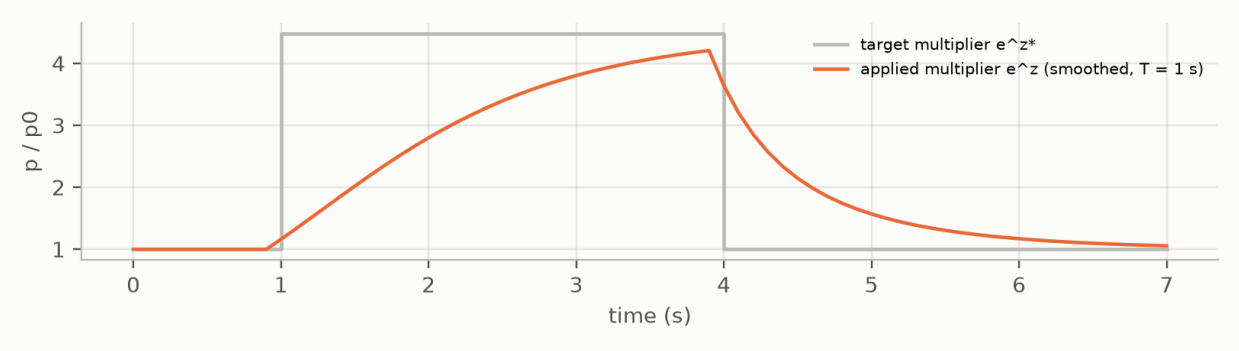


Figure 4. Smoothing: the target jumps to ×4.5 at t = 1 s and back at t = 4 s; the applied multiplier follows within about a second.

One control step, number by number

Take the ego cruising at 15 m/s in light traffic. Then a car starts closing from the rear-left with a time to contact of 1 s. The features change from the calm values to the threat values below (all others 0):

feature	calm	car closing rear-left
ttc_rear_side	0.00	0.63
speed	0.44	0.44
density	0.26	0.39

Target exponent z* = clip(W·f + b) and the resulting multiplier p/p0 (after the absolute limits), using the trained weights. The contributions column lists $W_{p,f} \times f$ for the non-zero features:

parameter	offset b	feature contributions W·f	z*	×p0	value
λ_obs	-0.14	ttc_rear_side +1.27, speed +0.33, density +0.13	+1.58	×4.88	929.6
τ_y	-0.18	ttc_rear_side -0.39, speed +0.66, density +0.04	+0.14	×1.15	0.4332
α	-1.04	ttc_rear_side +0.66, speed -0.46, density -0.60	-1.43	×0.48	0.01
radius	+0.92	ttc_rear_side +0.37, speed -0.07, density -0.29	+0.92	×2.52	245.7

Smoothing then moves the applied value from the calm setting towards this target, 10 % of the remaining gap per step:

parameter	calm	after 0.1 s	after 0.5 s	after 1 s	after 2 s	target
λ_obs	×1.31	×1.49	×2.24	×3.08	×4.16	×4.88
τ_y	×1.67	×1.61	×1.43	×1.31	×1.20	×1.15
α	×0.48	×0.48	×0.48	×0.48	×0.48	×0.48
radius	×1.93	×1.98	×2.15	×2.30	×2.44	×2.52

Within one second the repulsion from every vehicle rises from ×1.3 to about ×3 of the fixed set, and it keeps rising towards ×4.9 while the threat lasts. The ego is pushed away from the car closing behind it. The lateral reach narrows slightly and the look-ahead grows. Once the car has passed, the features drop and λ_{obs} relaxes again over the next second or two. α does not move: its value $e^{z} \cdot p0$ is below the 0.01 floor in both situations, so it stays at the floor (×0.48 of p0).

7. What the policy learned

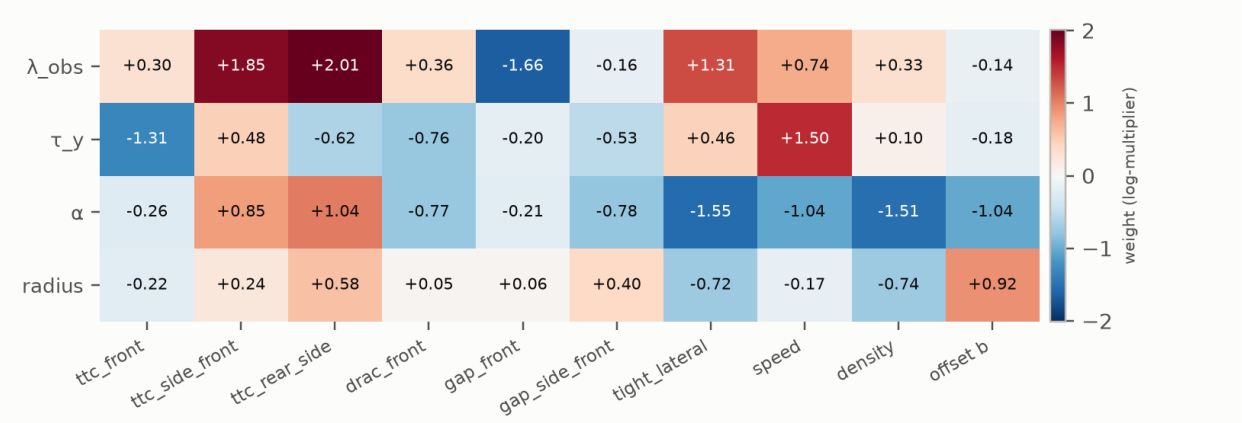


Figure 5. The 40 learned numbers. Each weight is a log-multiplier per unit feature: +1 means the parameter is multiplied by e ≈ 2.7 when that feature goes from 0 to 1.

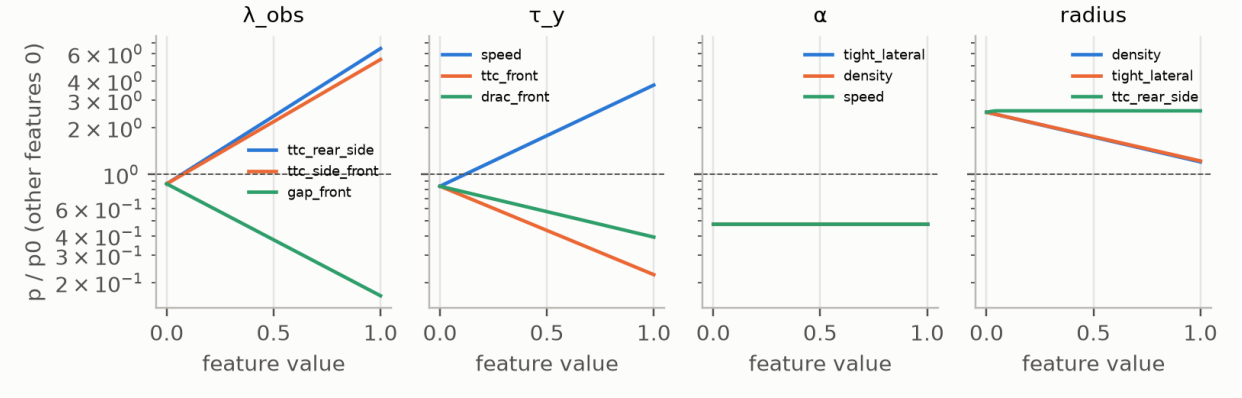


Figure 6. Each parameter's multiplier as one of its three strongest features rises from 0 to 1 (all other features 0, offset included). Real contexts combine several features.

Read together, the weights describe one strategy: **relaxed by default, sharp reactions to threats from the side and behind**.

λ_{obs}, repulsion strength

- Calm offset ×0.87.
- Rises with something closing from behind or the side (ttc_rear_side +2.0) or diagonally ahead (ttc_side_front +1.85), with little lateral room (+1.3) and with speed (+0.7).
- Falls when a vehicle is close ahead but not closing (gap_front −1.7): the ego follows calmly instead of being pushed sideways by a car that is no threat.

τ_y, lateral reach

- Rises with speed (+1.5): at 20 m/s the lateral influence zone is about twice as wide.
- Falls when closing in ahead (ttc_front −1.3): vehicles beside the ego count less, leaving room to swerve around the obstacle ahead.

α, velocity dependence of the reach

- The offset (×0.35) puts it below its 0.01 floor in calm traffic, and the strongest weights (density −1.5, tight_lateral −1.6, speed −1.0) push it further down. In practice α is pinned at its lower limit in 100% of control steps (Figure 8): the policy effectively chose the smallest allowed α, i.e. obstacles' reach barely depends on relative speed.

Obstacle radius, look-ahead

- Offset ×2.51: the policy looks much further ahead than the fixed set (97 m). Measured median ×1.78 (≈ 174 m).

- Shorter in dense traffic and when lateral room is tight (−0.7 each), so distant vehicles do not distract it in a crowd.

8. The policy in action

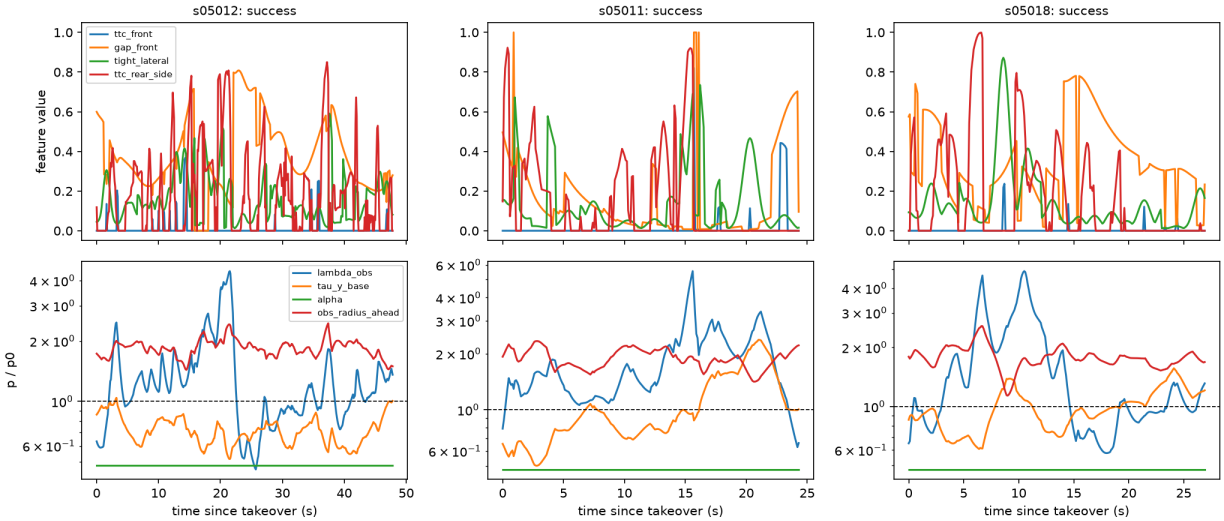


Figure 7. Three test runs (all successes). Top: four of the features. Bottom: the parameter multipliers p/p0 (log scale). λ_obs (blue) spikes to ×3–5 when ttc_rear_side or tight_lateral rise and relaxes afterwards; α (green) is flat at its 0.01 floor.

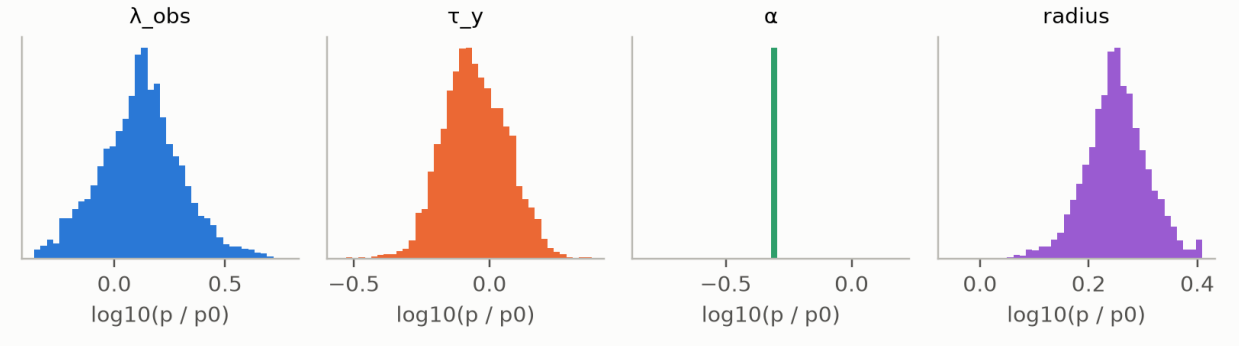


Figure 8. How far the parameters move: all 9,992 control steps of 40 test runs.

parameter	median	5 % … 95 %	steps at a limit
λ_obs	×1.33	×0.64 … ×2.71	0%
τ_y	×0.87	×0.58 … ×1.41	0%
α	×0.48	×0.48 … ×0.48	100%
radius	×1.78	×1.43 … ×2.21	1%

Worked examples

Multipliers computed with the real policy code for typical situations (target values, before smoothing; features not listed are 0):

situation	λ_obs	τ_y	α	radius
Calm: nothing near, 15 m/s, light traffic	×1.31	×1.67	×0.48	×1.93
Following a car 10 m ahead at equal speed	×0.50	×1.50	×0.48	×1.81
Car closing from rear-left, TTC 1 s	×4.88	×1.15	×0.48	×2.52
Diagonal car ahead closing, TTC 1 s, tight sideways	×9.21	×2.15	×0.48	×1.52
Closing on the car ahead, TTC 1.5 s, braking 2 m/s²	×0.76	×0.56	×0.48	×1.66

The same ego behaves differently in each. Behind a non-threatening car repulsion is lowered, so it follows calmly. When a vehicle closes from the side or behind, repulsion jumps several-fold (×5–9), pushing the ego away or slowing it until the threat passes. Closing on the

car directly ahead is different: repulsion stays slightly *below* the fixed set (the 'vehicle close ahead' weight outweighs the closing weights) and the lateral reach narrows. The policy leaves head-on closing mostly to the field's existing shape and concentrates its reactions on side and rear threats.

9. How it was trained

There is no correct parameter value at time t to learn from (many values work and a choice shows its effect only seconds later), so the policy was trained directly on what matters:

```
J(W, b) = success rate + 0.1 · mean progress
          over 100 train scenarios × 2 futures, takeover (x ≈ 100 m) → x = 400 m
```

CMA-ES (an evolution strategy) searched the 40 numbers, starting at 0 (= the fixed set): 30 generations × 15 candidates, every candidate scored on the same 200 runs. Every 4 generations the search mean was scored on 100 validation scenarios × 3 futures, and the final policy is the validated one with the highest J: the search mean of generation 12 (validation success 0.730 vs 0.647 for the fixed set). The fixed set itself was trained by the same CMA-ES over all 7 parameters with the same budget (450 candidates).

Why not supervised learning or gradient-based reinforcement learning?

- There are no labels: no one knows the 'right' λ_{obs} at a given moment.
- The objective is a success rate from a simulator. It is not differentiable, and it is flat almost everywhere: a small weight change usually changes no outcome, then suddenly flips one run.
- There are only 40 numbers. For this size, black-box search that only needs a score per candidate is simple and reliable. Evolution strategies such as CMA-ES are a standard choice for direct policy search with small policies.

How CMA-ES works here

```
start:  mean m = 0 (the fixed set),  step size σ = 0.5,  covariance C = I
repeat for 30 generations:
  1. sample 15 candidates   x_i = m + σ · N(0, C)            (40 numbers each)
  2. score each             J(x_i) on the same 200 train runs  (100 scenarios × 2
futures)
  3. rank; new mean m       = weighted mean of the best 7
  4. adapt C                towards the directions the best candidates came from
  5. adapt σ                larger if successive steps agree, smaller if they cancel
  every 4 generations: score the mean m on 300 validation runs; keep the best
```

- **Ranks, not scores.** Only the order of the candidates is used, so the absolute scale of J does not matter, and a single lucky run cannot dominate.
- **Covariance.** C learns which weights should change together (for example two features that measure similar dangers), so later samples are drawn along useful directions.
- **Same traffic for every candidate.** All 15 candidates drive the same 200 runs with the same SUMO seeds, and runs are exactly repeatable. Differences between candidates therefore come only from the weights, not from traffic luck. This is what makes a 200-run score usable for ranking.
- **Sign constraint.** For the three constrained weights, the optimiser's raw number x is used as |x|. The search space stays unconstrained, but the policy can only increase λ_{obs} with ttc_front, ttc_side_front and drac_front.
- **Selection on validation, not training.** The training score of the best candidate is optimistic: it is the best of 15 on the same runs. The search mean, scored on separate validation scenarios, is the honest estimate, and the final policy is the mean with the best

validation score.

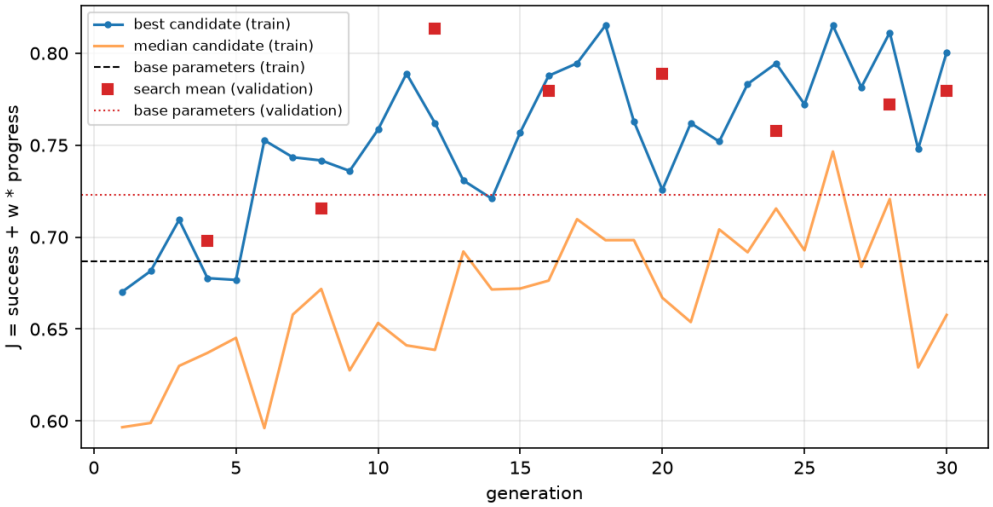


Figure 9. Training progress. Squares: the search mean on validation (the selection criterion); dotted: the fixed set on validation. Training J is optimistic (best of 15).

10. Results on the training road

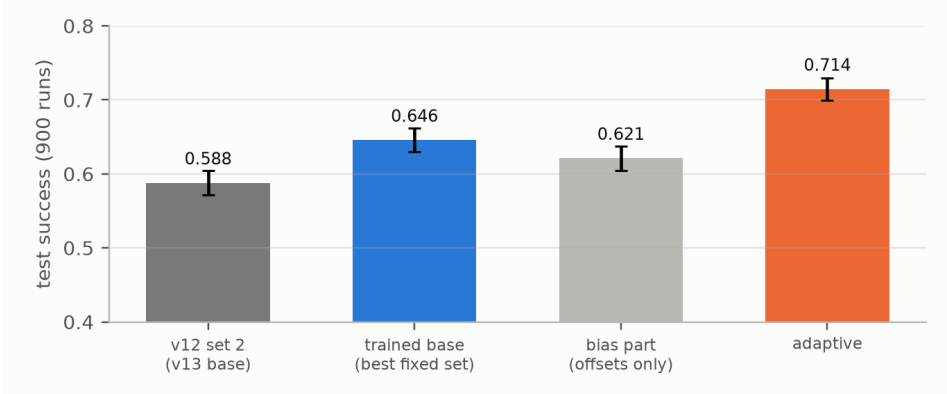


Figure 10. Test success, 300 fresh scenarios × 3 futures (±1 standard error).

comparison (same 900 runs)	difference	only A / only B	McNemar p
adaptive vs trained fixed set	+6.9 ± 2.2	182 / 120	0.00043
trained fixed set vs v12 set 2	+5.8 ± 2.5	206 / 154	0.0071
adaptive vs its offsets only	+9.3 ± 2.4	201 / 117	2.9e-06
offsets only vs trained fixed set	-2.4 ± 2.1	127 / 149	0.21
adaptive vs v12 set 2	+12.7 ± 2.3	219 / 105	2.3e-10

How to read the table. Every controller drove the same 900 runs, so the results are compared run by run. Runs that both controllers pass or both fail say nothing about which is better. Only the runs where they disagree count: 'only A' is the number of runs only the first one passed, 'only B' the reverse. **McNemar's test** asks how likely a split at least this uneven would be if both controllers were equally good (each disagreement a coin flip). For adaptive vs the fixed set, 182 / 120 out of 302 disagreements has p = 0.0004: very unlikely by chance. The difference column is the change in success rate in points, with its standard error. **Offsets only** is the trained policy with all feature weights set to zero: it keeps the learned calm setting but never adapts.

- **Adapting helps beyond tuning.** The fixed set had the same search, budget and all 7 parameters, and the adaptive policy still beats it by 6.9 points.
- **The gain is the adapting itself.** The policy's offsets without the feature terms are no better than the fixed set (−2.4, not significant).

- **Part of v13's earlier +16 points came from a weak base:** tuning the fixed set alone gained 5.8 points over v12 set 2.
- **Selection held up:** validation 0.730, test 0.714; the small drop is the usual effect of picking the best validation checkpoint.

11. Results on a 1000 m road (no retraining)

Both controllers were trained on a 300 m segment (x ≈ 100 → 400 m). To test whether the policy generalises, the same trained base and policy drove about 1000 m on a 1250 m road (100 new scenarios × 2 futures, 200 s limit), with no retraining.

controller (≈ 1000 m)	success	progress
v12 set 2	0.315	0.513
trained fixed set	0.345	0.528
adaptive policy	0.460	0.629

Adaptive vs the fixed set: **+11.5 points** (only adaptive 54, only fixed 31; McNemar p = 0.017). The advantage holds over the longer drive. Every controller fails more often over 1000 m because there are more chances to meet an unavoidable situation, but the adaptive policy keeps a clear lead at every distance (Figure 12).

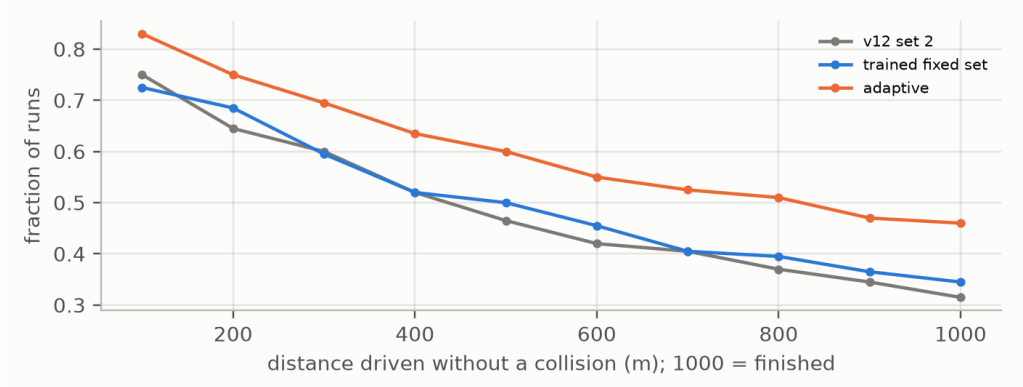


Figure 12. Fraction of runs still collision-free after each distance on the 1000 m road. The rightmost point is the fraction that finished.

12. Limits and next steps

- **α does not really adapt:** it sits at its 0.01 floor. Either the floor should be lowered, or α could be fixed and dropped from the adapted set.
- **Trained on 300 m only.** It was tuned on the segment x ≈ 100 → 400 m. It still beats the fixed set over 1000 m (Section 11), but success there is lower for everyone; training on longer segments may help.
- **Unavoidable failures stay.** About 29 % of runs still fail; some are situations no parameter set can handle, where only something outside the potential (a safety filter, a learned acceleration correction) could help.
- **Small by design.** 40 weights, smooth and bounded, so it can be read and checked. Next steps if more is needed: adapt all 7 parameters, a tiny MLP trained the same way, or more features.
- **Repeatability.** Every run is exactly repeatable (SUMO launched without address randomisation and with a folder-independent command line), so all policies were compared on identical traffic.

13. Reproducing

step	command (v14/…)	time (7 jobs)
fixed set	`base: step2_train_base.py, step3_evaluate.py`	~6 h
adaptive	`adaptive: step2_train_policy.py, step3_evaluate.py, step4_analyze.py`	~7.5 h
1000 m test	`longroad: evaluate_longroad.py`	~35 min
watch a run	`adaptive: dipf_controller.py --episode … --policy policy.json --gui`	–

14. Glossary

term	meaning
DIPF	the potential-field controller: the ego follows the clamped negative gradient of U
p0, fixed set	the best constant parameter values, found by CMA-ES in v14/base
multiplier p/p0	how far the policy moved a parameter from the fixed set; ×1 = unchanged
scenario	a saved SUMO state at the takeover moment (all vehicles, the ego, RNG state)
future	one random continuation of a scenario, set by the SUMO seed (1, 2, 3)
success	ego reaches x = 400 m (1100 m on the long road) without a collision within the time limit
progress	fraction of the segment covered before the run ended
J	training objective: success rate + 0.1 × mean progress
TTC	time to contact between the two bodies at constant velocities
DRAC	deceleration needed to avoid contact with the car ahead
CMA-ES	covariance matrix adaptation evolution strategy: a black-box optimiser that samples, ranks and adapts a Gaussian search distribution
McNemar test	paired test on the runs where two controllers disagree
offsets only / bias part	the trained policy with W = 0: learned calm setting, no adapting